

React JS — Detailed Lecture Notes

Sessions: Introduction to React JS + JSX & Components

These notes are designed so that a student can **understand the theory, set up React on their computer, create a complete project, execute every practical, and complete assignments independently.**

Session 1: Introduction to React JS

1. What is React?

React is a JavaScript library used to build **user interfaces (UI)**, especially for web applications.

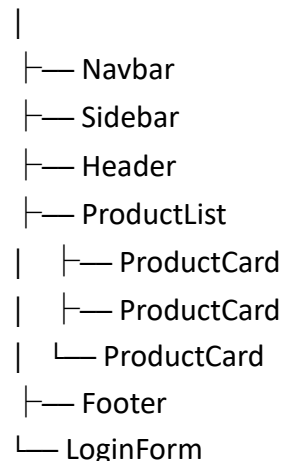
React was originally developed by Facebook and is now maintained by Meta and the open-source community.

Simple definition

React is a JavaScript library for building interactive and reusable user interfaces using components.

For example, a website can be divided into:

Website



Each part can become a **React component**.

2. Why Do We Need React?

Before React, developers commonly used JavaScript to directly manipulate HTML.

For example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Example</title>
</head>
<body>

<h1 id="heading">Hello</h1>
```

```
<button onclick="changeText()">Change</button>
```

```
<script>
```

```
function changeText() {  
  document.getElementById("heading").innerText = "Welcome to React";  
}
```

```
</script>
```

```
</body>
```

```
</html>
```

This works, but as applications become large, managing the DOM manually can become difficult.

React provides a component-based approach.

3. React Features

Important React features include:

1. Component-Based Architecture

Applications are divided into reusable components.

App

|

├─ Navbar

├─ Home

├─ Product

└─ Footer

2. JSX

JSX allows us to write HTML-like syntax inside JavaScript.

```
const element = <h1>Hello React</h1>;
```

3. Reusable Components

We can create one component and use it multiple times.

```
function Welcome() {  
  return <h2>Welcome Student</h2>;  
}
```

Then:

```
<Welcome />
```

```
<Welcome />
```

```
<Welcome />
```

4. Declarative UI

In React, we describe **what the UI should look like**.

Instead of manually telling JavaScript:

Find element

↓

Change text

↓

Change class

↓

Update HTML

we describe the desired UI.

5. Virtual DOM

React uses a virtual representation of the UI to efficiently determine what needs to be updated.

For beginners, remember:

React minimizes unnecessary DOM updates by comparing UI changes before updating the real DOM.

6. One-Way Data Flow

Data normally flows from parent components to child components.

Parent

↓

Child

↓

Grandchild

This makes application behavior easier to understand.

7. Large Ecosystem

React works with many other tools and libraries.

Examples:

React

|

├— React Router

├— Redux

├— React Query

├— Axios

├— Tailwind CSS

├— Vite

└— Testing libraries

4. What is an SPA?

SPA means:

Single Page Application

In a traditional website, navigating between pages can involve loading another HTML document.

Example:

Home



Server



home.html

About



Server



about.html

In an SPA:

Browser



Single HTML page



JavaScript / React

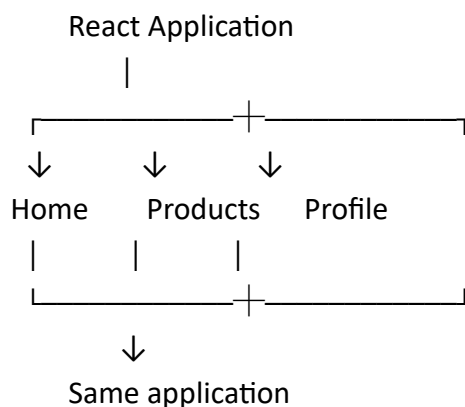


UI changes dynamically

The browser can update the displayed content without completely reloading the page.

5. Example of an SPA

Consider an online shopping application:



The UI changes based on the user's interaction.

6. Advantages of SPA

Faster navigation

Only necessary UI portions need to change.

Better user experience

The application feels more like a desktop/mobile application.

Component reuse

The same UI components can be reused.

Easier state management

React provides patterns and libraries for managing application data.

7. React vs JavaScript

This is an important topic for students.

JavaScript

Programming language

Can manipulate DOM directly

Can build websites

No component system built into language

Can work without React

General-purpose

React

JavaScript library

Uses React's UI rendering model

Helps build component-based UIs

Components are central

React requires JavaScript

Primarily focused on UI

Important

React is **not a replacement for JavaScript**.

React is built using JavaScript.

Therefore students should understand:

JavaScript



React



React Applications

8. React Ecosystem

The React ecosystem contains tools used with React applications.

React

Used for building UI.

React Router

Used for navigation/routing.

Redux

Used for state management.

Axios

Used for making HTTP requests.

Vite

Used for creating and developing React projects.

Tailwind CSS

Used for styling.

React Testing Library

Used for testing React components.

9. Node.js and npm

To work with modern React projects, students commonly use **Node.js** and **npm**.

What is Node.js?

Node.js allows JavaScript to run outside the browser.

Normally:

JavaScript

↓

Browser

With Node.js:

JavaScript

↓

Node.js

↓

Computer / Server

10. What is npm?

npm means:

Node Package Manager

It is used to:

- install packages
- manage dependencies
- run project scripts
- maintain project configuration

Example:

npm install

Another example:

npm run dev

11. Check Node.js Installation

Open VS Code terminal or PowerShell.

Run:

node -v

Example:

v22.x.x

Then:

npm -v

Example:

10.x.x

The exact version may be different.

12. What is Vite?

Vite is a modern development tool used to create and run frontend projects.

It provides:

- fast development server
- fast startup
- project scaffolding
- production build support
- modern JavaScript tooling

For React beginners, Vite is a convenient way to create a React project.

13. Create React Project Using Vite

Open PowerShell or VS Code terminal.

Go to the folder where you want to create your project.

Example:

cd Desktop

Create project:

npm create vite@latest react-learning

Vite will ask questions.

Choose:

Framework: React

Variant: JavaScript

Then enter the project:

cd react-learning

Install dependencies:

npm install

Start development server:

npm run dev

You will get a local URL similar to:

http://localhost:5173/

Open that URL in your browser.

14. Complete Project Creation Commands

Students can remember the basic workflow:

npm create vite@latest react-learning

cd react-learning

npm install

npm run dev

15. React Project Structure

A Vite React project generally looks similar to:

react-learning/

```
|
| └─ node_modules/
|
| └─ public/
|
| └─ src/
|   └─ assets/
|   └─ App.css
|   └─ App.jsx
|   └─ index.css
|   └─ main.jsx
|
| └─ .gitignore
| └─ eslint.config.js
| └─ index.html
| └─ package.json
| └─ package-lock.json
└─ vite.config.js
```

16. Important Folders and Files

node_modules

Contains installed packages.

Students normally **do not manually edit this folder**.

public

Contains static files that can be served directly.

Example:

```
public/
| └─ logo.png
└─ image.jpg
```

src

This is the main development folder.

Most React code is written here.

src/

```
├── components/  
├── App.jsx  
├── main.jsx  
└── index.css
```

17. package.json

package.json contains project information and dependencies.

Example structure:

```
{  
  "name": "react-learning",  
  "version": "0.0.0",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite",  
    "preview": "vite"  
  }  
}
```

It also contains dependencies.

18. main.jsx

main.jsx is the entry point of the React application.

A typical structure is:

```
import { StrictMode } from 'react';  
import { createRoot } from 'react-dom/client';  
import App from './App.jsx';
```

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

Understand the flow

```
main.jsx  
  ↓  
App.jsx  
  ↓  
Components  
  ↓  
UI
```

19. index.html

The Vite application has an HTML file containing the root element.

Example:

```
<div id="root"></div>
```

React renders the application into this element.

Conceptually:

index.html



```
<div id="root">
```



React



App



Components

20. First React Practical

Open:

src/App.jsx

Replace its contents with:

```
function App() {  
  return (  
    <div>  
      <h1>Hello React</h1>  
      <p>Welcome to my first React application.</p>  
    </div>  
  );  
}
```

```
export default App;
```

Save the file.

Your browser should automatically update.

Session 2 — JSX & Components

21. What is JSX?

JSX means:

JavaScript XML

JSX allows developers to write HTML-like syntax inside JavaScript.

Example:

```
const element = <h1>Hello React</h1>;
```

JSX is not exactly HTML.

It is syntax that is transformed into JavaScript.

22. JSX Example

```
function App() {  
  return (  
    <div>  
      <h1>Student Management</h1>  
      <p>Welcome to the application.</p>  
    </div>  
  );  
}
```

export default App;

Here:

<h1>Student Management</h1>
is JSX.

23. JSX Rule 1 — Return One Parent Element

Incorrect:

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

Correct:

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

24. JSX Fragments

Instead of creating an unnecessary <div>, we can use a fragment:

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </>  
);
```

<>...</> is called a fragment.

25. JSX Rule 2 — Close Every Tag

HTML sometimes allows certain flexible syntax, but JSX expects properly formed elements.

Correct:

```

```

Correct:

```
<input type="text" />
```

26. JSX Rule 3 — className Instead of class

HTML:

```
<div class="container">
```

JSX:

```
<div className="container">
```

Because class has a special meaning in JavaScript.

27. JSX Rule 4 — camelCase Attributes

HTML:

```
onclick=""
```

JSX commonly uses:

```
onClick=""
```

Examples:

```
className
```

```
onClick
```

```
onChange
```

```
tabIndex
```

28. JavaScript Expressions in JSX

We can write JavaScript expressions inside:

```
{ }
```

Example:

```
function App() {
```

```
  const name = "Amit";
```

```
  return (
```

```
    <div>
```

```
      <h1>Hello {name}</h1>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

Output:

Hello Amit

29. Mathematical Expression in JSX

```
function App() {  
  
  const a = 10;  
  const b = 20;  
  
  return (  
    <div>  
      <h2>Result: {a + b}</h2>  
    </div>  
  );  
}
```

export default App;

Output:

Result: 30

30. Multiple Expressions

```
function App() {  
  
  const name = "Amit";  
  const age = 23;  
  const city = "Delhi";  
  
  return (  
    <div>  
      <h1>{name}</h1>  
      <p>Age: {age}</p>  
      <p>City: {city}</p>  
    </div>  
  );  
}
```

export default App;

31. Conditional Expression in JSX

We can use the ternary operator.

```
function App() {
```

```
const isLoggedIn = true;

return (
  <div>
    <h1>
      {isLoggedIn ? "Welcome User" : "Please Login"}
    </h1>
  </div>
);
}

export default App;
```

32. What is a Component?

A component is an independent and reusable part of the user interface.

Example:

Website

```
|
├── Navbar
├── Header
├── Sidebar
├── ProductList
├── ProductCard
└── Footer
```

Each can be represented as a component.

33. Functional Components

Modern React primarily uses functional components.

Example:

```
function Welcome() {
  return <h1>Welcome to React</h1>;
}
```

```
export default Welcome;
```

Component name:

Welcome

Notice that component names should normally begin with an uppercase letter.

34. Using a Component

Suppose we have:

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}
```

We can use it as:

```
<Welcome />
```

35. Complete Component Practical

Create:

```
src/  
└─ components/  
    └─ Welcome.jsx
```

Welcome.jsx

```
function Welcome() {  
  return (  
    <div>  
      <h2>Welcome Student</h2>  
      <p>Start learning React JS.</p>  
    </div>  
  );  
}
```

```
export default Welcome;
```

Now modify App.jsx:

```
import Welcome from './components/Welcome';
```

```
function App() {  
  return (  
    <div>  
      <h1>React Learning App</h1>  
      <Welcome />  
    </div>  
  );  
}
```

```
export default App;
```

36. Understanding import/export

React applications contain many files.

For example:

App.jsx

Welcome.jsx

Header.jsx

Footer.jsx

We need a way to use code from one file in another.

JavaScript provides:

export

import

37. Named Export

File:

```
// Student.jsx
```

```
export function Student() {  
  return <h2>Student Component</h2>;  
}
```

Import:

```
import { Student } from "./Student";
```

Use:

```
<Student />
```

38. Default Export

File:

```
function Student() {  
  return <h2>Student Component</h2>;  
}
```

```
export default Student;
```

Import:

```
import Student from "./Student";
```

Then:

```
<Student />
```

39. Named vs Default Export

Default

```
export default Student;
```

Import:

```
import Student from "./Student";
```

Named

```
export function Student() {  
}
```

Import:

```
import { Student } from "./Student";
```

40. Multiple Components Practical

Create:

```
src/  
|  
├─ App.jsx  
|  
└─ components/  
    ├─ Header.jsx  
    ├─ Navbar.jsx  
    └─ StudentCard.jsx  
    └─ Footer.jsx
```

Header.jsx

```
function Header() {  
  return (  
    <header>  
      <h1>Student Management System</h1>  
    </header>  
  );  
}
```

```
export default Header;
```

Navbar.jsx

```
function Navbar() {  
  return (  
    <nav>  
      <a href="#">Home</a>{" "  
      <a href="#">Students</a>{" "  
      <a href="#">About</a>  
    </nav>  
  );  
}
```

```
export default Navbar;
```

StudentCard.jsx

```
function StudentCard() {  
  return (  
    <div>
```

```
        <h2>Amit Kumar</h2>
        <p>Course: React JS</p>
        <p>Marks: 85</p>
    </div>
  );
}
```

```
export default StudentCard;
```

Footer.jsx

```
function Footer() {
  return (
    <footer>
      <p>© 2026 Student Management System</p>
    </footer>
  );
}
```

```
export default Footer;
```

41. App.jsx

Now combine all components:

```
import Header from "./components/Header";
import Navbar from "./components/Navbar";
import StudentCard from "./components/StudentCard";
import Footer from "./components/Footer";
```

```
function App() {
  return (
    <div>
      <Header />
      <Navbar />
      <StudentCard />
      <Footer />
    </div>
  );
}
```

```
export default App;
```

42. Component Tree

The application now looks like:

App

|

├─ Header

├─ Navbar

├─ StudentCard

└─ Footer

This is one of the most important ideas in React.

43. Reusable Components

The biggest advantage of components is **reuse**.

Instead of creating:

StudentCard1

StudentCard2

StudentCard3

StudentCard4

we create one:

StudentCard

and reuse it.

For example:

```
<StudentCard />
```

```
<StudentCard />
```

```
<StudentCard />
```

Later, we will make each card display different information using **props**.

44. Reusable Product Card Practical

Create:

src/components/ProductCard.jsx

Code:

```
function ProductCard() {  
  return (  
    <div>  
      <h2>Wireless Mouse</h2>  
      <p>Price: ₹599</p>  
      <button>Buy Now</button>  
    </div>  
  );  
}
```

```
export default ProductCard;
```

Use it:

```
import ProductCard from "../components/ProductCard";
```

```
function App() {  
  return (  
    <div>  
      <h1>Online Store</h1>  
  
      <ProductCard />  
      <ProductCard />  
      <ProductCard />  
    </div>  
  );  
}
```

```
export default App;
```

At this point all three cards contain the same information.

This demonstrates reuse.

In the upcoming React topics, we can pass different information into each card using **props**.

45. Practical: Student Profile Components

Create:

```
src/  
├── App.jsx  
└── components/  
    ├── StudentHeader.jsx  
    ├── StudentInfo.jsx  
    └── StudentFooter.jsx
```

StudentHeader.jsx

```
function StudentHeader() {  
  return (  
    <div>  
      <h1>Student Profile</h1>  
    </div>  
  );  
}
```

```
export default StudentHeader;
```

StudentInfo.jsx

```
function StudentInfo() {  
  return (  
    <div>
```

```
        <h2>Amit Kumar</h2>
        <p>Course: Computer Science</p>
        <p>Semester: 6</p>
    </div>
  );
}
```

```
export default StudentInfo;
```

StudentFooter.jsx

```
function StudentFooter() {
  return (
    <div>
      <p>End of Student Profile</p>
    </div>
  );
}
```

```
export default StudentFooter;
```

App.jsx

```
import StudentHeader from "../components/StudentHeader";
import StudentInfo from "../components/StudentInfo";
import StudentFooter from "../components/StudentFooter";
```

```
function App() {
  return (
    <div>
      <StudentHeader />
      <StudentInfo />
      <StudentFooter />
    </div>
  );
}
```

```
export default App;
```

46. Practical: JSX Student Report

Create:

src/App.jsx

```
function App() {
```

```
  const studentName = "Amit";
```

```

const marks = 85;
const subject = "React JS";

return (
  <div>
    <h1>Student Report</h1>

    <p>Name: {studentName}</p>
    <p>Subject: {subject}</p>
    <p>Marks: {marks}</p>

    <p>
      Result:
      {marks >= 40 ? " Pass" : " Fail"}
    </p>

    <p>
      Grade:
      {marks >= 90
        ? " A+"
        : marks >= 80
        ? " A"
        : marks >= 70
        ? " B"
        : " C"}
    </p>
  </div>
);
}

```

export default App;

This practical combines:

- JSX
- JavaScript variables
- expressions
- conditional rendering
- ternary operator

47. Common Beginner Errors

Error 1 — Component starts with lowercase

Avoid:

```
function studentCard() {  
}
```

Prefer:

```
function StudentCard() {  
}
```

Error 2 — Forgetting export

If you create:

```
function Header() {  
  return <h1>Header</h1>;  
}
```

and want to import it into another file, export it:

```
export default Header;
```

Error 3 — Incorrect import

If using default export:

```
export default Header;
```

use:

```
import Header from "./components/Header";
```

For named export:

```
export function Header() {  
}
```

use:

```
import { Header } from "./components/Header";
```

Error 4 — Multiple top-level JSX elements

Incorrect:

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

Correct:

```
return (  
  <>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </>  
);
```

Error 5 — Using class

Incorrect:

```
<div class="box">
```

Correct:

```
<div className="box">
```

48. Complete Mini Project — React Student Portal

Students can create the following project as the final practical.

Folder Structure

react-student-portal/

```
|
| └─ public/
|
| └─ src/
|   |
|   | └─ components/
|   |   | └─ Header.jsx
|   |   | └─ Navbar.jsx
|   |   | └─ StudentCard.jsx
|   |   └─ Footer.jsx
|   |
|   | └─ App.jsx
|   | └─ App.css
|   | └─ index.css
|   └─ main.jsx
|
| └─ package.json
└─ vite.config.js
```

Header.jsx

```
function Header() {
  return (
    <header>
      <h1>React Student Portal</h1>
      <p>Welcome to the Student Management System</p>
    </header>
  );
}
```

```
export default Header;
```

Navbar.jsx

```
function Navbar() {
  return (
    <nav>
      <a href="#">Home</a>{" | "}
      <a href="#">Students</a>{" | "}
      <a href="#">Courses</a>{" | "}
      <a href="#">Contact</a>
    </nav>
  );
}
```

```
export default Navbar;
```

StudentCard.jsx

```
function StudentCard() {
  return (
    <div>
      <h2>Amit Kumar</h2>
      <p>Course: Computer Science</p>
      <p>Marks: 85</p>
      <p>Status: Active</p>
    </div>
  );
}
```

```
export default StudentCard;
```

Footer.jsx

```
function Footer() {
  return (
    <footer>
      <p>© 2026 React Student Portal</p>
    </footer>
  );
}
```

```
export default Footer;
```

App.jsx

```
import Header from "../components/Header";
import Navbar from "../components/Navbar";
```

```
import StudentCard from "../components/StudentCard";
import Footer from "../components/Footer";

function App() {
  return (
    <div>
      <Header />

      <Navbar />

      <main>
        <StudentCard />
        <StudentCard />
        <StudentCard />
      </main>

      <Footer />
    </div>
  );
}

export default App;
```

49. What Students Learned

After these two sessions, students should understand:

React Fundamentals

- What React is
- Why React is used
- React features
- SPA
- React vs JavaScript
- React ecosystem
- Node.js
- npm
- Vite
- React project creation
- Project structure

JSX

- JSX meaning
- JSX syntax
- JSX rules

- Parent elements
- Fragments
- className
- JSX expressions
- JavaScript inside {}

Components

- Component concept
- Functional components
- Component naming
- Component files
- Import/export
- Default exports
- Named exports
- Reusable components
- Component tree

50. Assignments

Assignment 1 — React Introduction

Create a React application called:

college-portal

It should display:

Welcome to ABC College

Courses:

BCA

B.Tech

MCA

MBA

Use JSX.

Assignment 2 — JSX Expressions

Create a student report containing:

Student Name

Age

Course

Marks

Percentage

Pass/Fail

Use JavaScript variables and JSX expressions.

Assignment 3 — Component Creation

Create these components:

Header

Navbar

Home

About

Contact

Footer

Combine all components inside App.jsx.

Assignment 4 — Product Components

Create a reusable:

ProductCard

component.

Display at least five products.

Each card should contain:

Product Name

Price

Category

Availability

Challenge: For now, create the component structure manually. In the next topic, use props to make each card display different data.

Assignment 5 — Employee Profile

Create:

EmployeeHeader

EmployeeInfo

EmployeeSalary

EmployeeFooter

Display:

Employee Name

Employee ID

Department

Designation

Salary

51. Final Assignment — Student Management UI

Create a React application with this structure:

App

|

└─ Header

└─ Navbar

```
└─ StudentList
|  └─ StudentCard
|  └─ StudentCard
|  └─ StudentCard
└─ Footer
```

Each student card should contain:

Student Name

Roll Number

Course

Marks

City

Result

Use:

- Vite
- React
- JSX
- Functional components
- Import/export
- Reusable components
- JavaScript expressions
- Conditional expressions

Expected UI concept

```
=====
      STUDENT MANAGEMENT SYSTEM
=====
```

[Home](#) | [Students](#) | [About](#) | [Contact](#)

Student: Amit

Roll No: 101

Course: B.Tech CSE

Marks: 85

Result: PASS

Student: Rahul

Roll No: 102

Course: BCA

Marks: 72

Result: PASS

Student: Priya

Roll No: 103

Course: MCA

Marks: 35

Result: FAIL

© 2026

=====

52. Important Questions for Viva/Interview

Basic

Q1. What is React?

React is a JavaScript library for building user interfaces.

Q2. Who developed React?

React was originally developed by Facebook.

Q3. Is React a programming language?

No. React is a JavaScript library.

Q4. What is JSX?

JSX is a syntax that allows us to write HTML-like UI structures inside JavaScript.

Q5. What is a component?

A component is a reusable part of a React application's UI.

Q6. What is a functional component?

A JavaScript function that returns React UI is called a functional component.

Q7. What is Vite?

Vite is a modern frontend development and build tool commonly used to create React projects.

Q8. What is npm?

npm is the Node Package Manager used to install and manage JavaScript packages.

Q9. What is SPA?

SPA stands for Single Page Application, where the application can update its UI dynamically without requiring a full page reload for every navigation.

Q10. Why do React components start with uppercase letters?

Uppercase naming helps React distinguish user-defined components from native HTML elements.

Q11. What is export default?

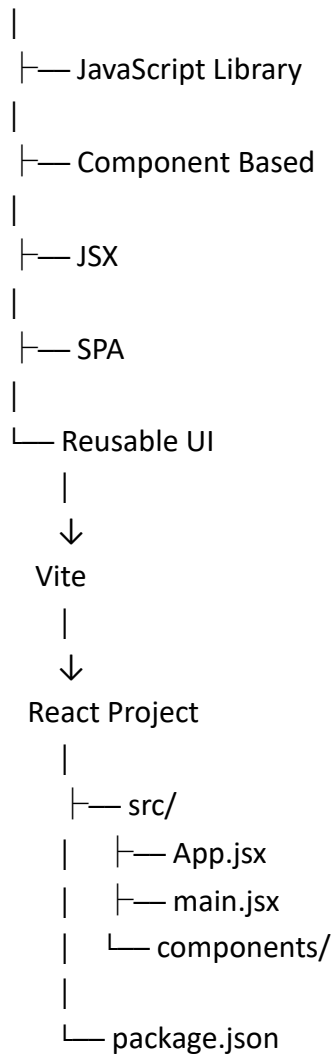
It exports one default value from a JavaScript module.

Q12. What is import?

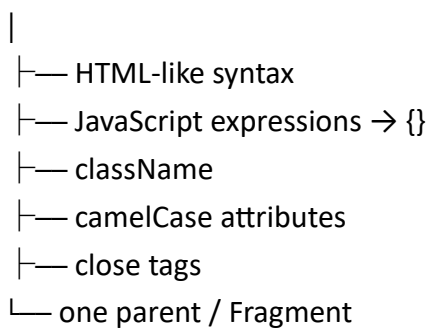
It allows code exported from another module to be used in the current module.

53. One-Page Revision

REACT JS



JSX



COMPONENT

- |
- ├— Functional Component
- ├— Reusable
- ├— Import
- ├— Export
- └— Component Tree

Recommended classroom practical sequence

First 30 minutes: React, SPA, ecosystem, Node/npm, Vite

Next 30 minutes: Create Vite project + understand project structure

Next 30 minutes: JSX rules + expressions + first practicals

Final 30 minutes: Functional components + import/export + reusable components + mini project

This gives students both the **conceptual foundation and enough hands-on practice** before moving to the next React topics such as **props, events, state, conditional rendering, lists, forms, and CRUD**.